

Cost and Accuracy of Long-Term Memory in Distributed Multi-Agent Systems Based on Large Language Models

Benedict Wolff

*School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology
Stockholm, Sweden
bjpwolff@kth.se
bjpw@live.de*

Jacopo Bennati

*School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology
Stockholm, Sweden
jbennati@kth.se
jacobbista.bennati@gmail.com*

Abstract—Long-term memory (LTM) is fundamental to large language model (LLM)-based agents in the emerging internet of agents (IoA), where distributed multi-agent systems (DMASs) span cloud and edge networks. Existing evaluations are typically published by framework providers and focus on token usage and latency, rarely accounting for system-level cost or deployment in DMAS. These gaps are addressed with an independent reproducible testbed that evaluates accuracy, latency, CPU time, peak RAM, disk I/O and network usage in a simulated cloud-edge environment. Three venture capital-funded frameworks spanning vector, graph, and hybrid architectures, namely mem0, Graphiti, and cognee, are compared alongside retrieval-augmented generation (RAG) and full-context baselines on the LoCoMo benchmark under unconstrained and constrained network scenarios. Two clusters emerge: mem0, RAG, and full-context reach 77% to 81% accuracy, while Graphiti and cognee reach only 55% to 56%, a gap driven by retrieval incompleteness rather than reasoning failure. The RAG baseline matches the upper cluster at 8.4 times lower total cost of ownership (TCO) than mem0, and both are the only non-dominated backends on the Pareto frontier. Latency and bandwidth constraints as well as jitter leave retrieval quality unchanged for every backend, while vector-based LTM incurs a modest latency penalty of 4% to 5% under edge-cloud constraints. Compression precision rather than context volume determines LTM accuracy, as full-context forwarding underperforms mem0 despite supplying the entire conversation for each question.

Index Terms—long-term memory, multi-agent system, internet of agents, retrieval-augmented generation, knowledge graph

Open-source research: the testbed, results, and notebooks for analysis and evaluation are available at: <https://github.com/wolffbe/dmas-memory>

I. INTRODUCTION

Large language model (LLM)-based agents are increasingly capable of decomposing goals into executable steps using in-context learning, integrating external tools and data sources as well as persisting thoughts using long-term memory (LTM) [1].

LTM refers to the ability of LLM-based agents to store, organize, and retrieve information across interactions, enabling persistent context, behavioral adaptation, and personalization beyond single-turn reasoning [2]. Retrieval-augmented

generation (RAG) is foundational to LTM by embedding overlapping chunks of data into high-dimensional vector representations [3]. However, this static approach struggles with dynamic knowledge integration, introduces noise, and overlooks structural relationships. LTM extends this paradigm into dynamic, agent-specific memory that accumulates over time, organized through textual, vector-based, graph-based and hybrid architectures, and supported by operations for data and knowledge indexing, retrieval, updating, and consolidation.

To evaluate these capabilities, LTM benchmarks such as LoCoMo and LongMemEval have been proposed, focusing on tasks including multi-session reasoning, information retrieval, temporal consistency, and knowledge updates [4], [5].

Despite advances in LTM, single-agent systems remain limited in addressing large-scale problems requiring distributed reasoning and coordination. This has led to the development of multi-agent systems (MASs), where specialized agents interact within a shared environment to enable collaborative decision-making, task delegation, and parallel problem solving [6].

Distributed multi-agent systems (DMASs) further reduce reliance on centralized orchestration by enabling MAS to coordinate across peer-to-peer networks, improving scalability, robustness, and privacy while avoiding bottlenecks associated with central controllers [7], [8]. The vision is an internet of agents (IoA) that supports networks of heterogeneous agents processing information through agent integration protocols, messaging architectures, and dynamic teaming, while maintaining local data sovereignty, and being applied in long-term knowledge work and scientific research [9]–[11].

Despite these developments, three gaps remain. First, existing evaluations of LTM are often not independent, as many comparisons are conducted or reported by the providers themselves. This introduces potential evaluation bias, since methods are often benchmarked against selective baselines under non-uniform conditions. Second, most studies focus on token-level performance metrics, while neglecting costs such as computation, network communication, and storage overhead introduced by LTM. Third, the performance of LTM in DMAS

remains underexplored, particularly in cloud-edge scenarios.

Filling these gaps is essential for further research on the IoA. The performance of LTM frameworks that agents in such a network build upon should be verifiable without any risk of evaluation bias. Given the wide range of technical constraints in network systems, understanding the resource consumption of LTM in different network scenarios is of equal importance.

For that reason, this research aims to contribute to the fields of LTM, DMAS and the IoA by answering the following three research questions (RQs):

- **RQ1:** What are the essential components of a uniform and independent testbed that enables reproducible comparison of different LTM frameworks?
- **RQ2:** How do the cost and accuracy of LTM frameworks perform in a DMAS compared to a MAS?
- **RQ3:** Which type of LTM framework offers the best trade-off between cost and accuracy in a DMAS?

In response to these RQs, this paper’s main contributions are:

- a testbed for evaluating LTM frameworks in a DMAS, measuring the accuracy and latency of responses as well as system-level costs in a simulated cloud-edge environment,
- independent benchmarks of three LTM frameworks funded by venture capital, namely mem0 (vector-based), Graphiti (graph-based), and Cognee (hybrid), including full-context and RAG baselines, using LoCoMo as the source of long-term conversational data, and
- evaluations under unconstrained and constrained network scenarios, capturing the trade-off between LTM accuracy, CPU time, peak RAM, disk I/O and network usage.

The rest of this paper is structured as follows. Section II reviews the related work. Section III presents the methodological approach to implementing and operating the testbed as well as analyzing its results described in Section IV. Section V contains the discussion, limitations, and future work.

II. RELATED WORK

This section reviews prior work on LLM-based agents from three complementary perspectives. LTM frameworks enable agents to memorize, organize, and retrieve knowledge across interactions. LTM benchmarks evaluate these frameworks against standardized tasks. The IoA addresses how agents interact and coordinate across peer-to-peer network infrastructures such as cloud-edge systems, extending MAS to DMAS.

Long-term Memory Frameworks

One of the earliest vector-based implementations of LTM is MemGPT, which augments the LLM with a hierarchical memory architecture inspired by virtual memory. It partitions the fixed-size context window into system instructions, writable working memory, and a FIFO conversation buffer with recursive summarization, while storing extracted information in an external vector-indexed memory that is retrieved through function calls when additional context is required [12].

Mem0 extracts salient facts via LLM calls against a rolling conversation summary, then reconciles them with semantically

similar entries in a vector store through tool-call operations that add, update, delete, or skip memories [13]. The mem0^g variant additionally encodes these facts as a directed labeled graph of entities and typed relations.

Among other graph-based systems, Zep builds upon Graphiti, a temporally-aware Neo4j engine that ingests messages as episodes, extracts entities and inter-entity facts as labeled edges via LLM prompts, and tracks a bitemporal validity window so that contradictory edges are invalidated rather than overwritten, while label-propagation clustering aggregates entities into a higher-level community subgraph [14]. Hindsight maintains four logical graphs: facts, experiences, entities, and beliefs, governed by retain, recall, and reflect operations [15].

Hybrid systems combine vector and graph primitives. Cognee combines a Neo4j graph with a Qdrant vector store through a modular Extract–Cognify–Load pipeline that chunks documents and applies schema-constrained LLM extraction to populate both stores with entities, relations, and summaries [16]. On the retrieval side, it exposes configurable strategies ranging from pure vector lookup over chunks to graph-completion routines that combine triplet retrieval with neighborhood traversal. Memoria pairs session-level summarization with a weighted knowledge graph that incrementally captures user traits, and behavioral patterns as structured entities and relationships [17].

These memory frameworks are almost universally benchmarked by their own authors, and the few cross-system comparisons among them run on heterogeneous harnesses, judges, and prompts, reducing evaluation to token consumption and task success. While latency is reported by several frameworks, CPU, RAM, disk I/O and network usage are rarely tracked.

Long-term Memory Benchmarks

LLM-based agent evaluation is organized along two axes [18]. Evaluation objectives cover agent behavior, capabilities, reliability, safety, and alignment. The evaluation process consists of interaction mode, evaluation data, metrics computation methods, evaluation tooling, and evaluation contexts. Evaluation data is further split into datasets, benchmarks, and domain specificity.

Latency, token usage, and cost are categorized as agent behaviour within evaluation objectives. Various benchmarks in this category show that evaluating the system-level cost and latency of agents is feasible. However, they are limited to GUIs or mobile agents rather than LTM, MAS or DMAS.

Memory and context retention within agent capabilities is further refined into memory spans and memory forms [19]. Memory spans include turn memory, conversation memory, and permanent memory, while memory forms cover textual form, such as complete, recent, retrieved and external interaction, and parametric form with fine-tuned memory and memory editing.

LoCoMo provides ten multi-session dialogues averaging 600 turns and 16,000 tokens over up to 32 sessions, with question categories spanning single-hop, multi-hop, temporal, open-domain, and adversary [4]. LongMemEval formalizes the five abilities information extraction, multi-session reasoning, temporal reasoning, knowledge updates, and abstention [5]. ES-MemEval extends this to emotional support, adding conflict

detection and user modelling [20]. MS-TOD addresses multi-session task-oriented dialogue with 132 personas and intent-aligned memory annotations [21]

Existing LTM benchmarks primarily evaluate memory accuracy and rarely cover system-level costs. Cost-aware agent benchmarks do not cover LTM, and neither line of work considers MAS or DMAS. To the best of our knowledge, this paper is the first to jointly evaluate LTM accuracy and system-level cost of a DMAS in a simulated cloud-edge scenario.

Internet of Agents

Active fields of research on the IoA include agent discovery, identity management, architectures, and edge deployments.

For agent discovery, a two-stage framework has been proposed in which agents first publish machine-interpretable capability descriptions and then perform context-aware search, ranking, and composition to assemble suitable agents for a given task [22]. The scheme integrates capability modelling, scalable and updatable indexing, and continual discovery.

Regarding identity management, LLM-based agents have been integrated with self-sovereign identity to mitigate prompt injection, unauthorized access, and API exploitation [23].

With respect to agent architectures, a DMAS underpinned by a blockchain and a trust-aware communication protocol has been proposed, providing verifiable interaction cycles, non-repudiation, and conditional confidentiality [24].

In the field of edge deployments, several lines of work address the gap between resource-constrained devices and the demands of LLM-based agents. CASK reduces inference-time memory in MAS through a dynamic sparse attention module and adaptive key value (KV)-cache compression, cutting memory usage by up to 40% while retaining over 95% of baseline accuracy on LongBench [25]. An adaptive placement and migration framework uses ant colony optimization and LLM-based decision making on top of AgentScope to reduce deployment latency and migration cost in dynamic edge environments [26]. Closest to the present work, a heterogeneous edge testbed on K3s, a lightweight Kubernetes distribution, with eight Jetson Nano and Raspberry Pi clients has been constructed, simulating realistic 4G/5G network conditions through Linux Traffic Control [27]. Quantization, asynchronous federated learning, distributed caching, and network-aware scheduling are evaluated on Llama 3 8B, with a reported 75% bandwidth reduction, 45% latency improvement, and 61% energy decrease per inference. The testbed methodology is closely related, but the focus lies on federated training and inference rather than evaluating LTM frameworks. LTM beyond the KV-cache is not considered.

Across these directions, no work to date evaluates LTM frameworks in DMAS or the IoA. Discovery, identity, and architecture work treat LTM as an internal property of individual agents. Edge-deployment work either compresses the KV-cache at inference time or focuses on training-time federated workloads. The testbed proposed in this paper fills this gap by evaluating accuracy and system-level cost of LTM frameworks under simulated conditions that resemble DMAS deployments.

III. METHOD

The methodology applied in this study follows the experimental framework described by Wohlin et al. [28], which structures empirical software engineering experiments into five phases, namely scoping, planning, operation, analysis and interpretation as well as presentation and packaging. The first four phases are described in this section. Presentation and packaging is realized through sections IV and V.

Scoping the Experiments

Scoping consists of defining the object of study, purpose, quality focus, perspective and context. The goal of the experiment is formulated using a structured Goal-Question-Metric template covering the object of study, purpose, quality focus, perspective, and context [29].

Object of Study: The objects of study are three LTM frameworks, all implemented as Python-based Docker services and selected to span the architectural spectrum from vector to graph storage: mem0, a vector-based system with LLM-driven compression that raised \$24 million USD [30]; Graphiti, a graph engine built on Neo4j released as the open-source core of Zep, which has raised \$2.3 million USD in pre-seed funding [31]; and cognee, a hybrid framework combining vector and graph architectures, which has raised \$7.5 million USD [32]. The candidate pool was restricted to LTM with both venture capital funding and published benchmarks, a set that also includes Hindsight and Letta, formerly known as MemGPT [12], [15]. From this pool, mem0, cognee, and Graphiti were selected for their comparable scale and intersecting technical stack consisting of Python, Docker, Neo4j and Qdrant. Hindsight and Letta were excluded as both market themselves as entire platforms for LLM-based agents rather than exclusively LTM. RAG and full-context implementations, where an LLM is given an entire conversation with each question, serve as baselines.

Purpose: The purpose of the experiment is to compare these frameworks under two consistent network scenarios, characterizing the trade-off between memorizing and retrieving knowledge as well as system-level resource consumption.

Quality Focus: Two quality attributes are considered. The first is memory accuracy, capturing whether an agent can correctly answer questions that depend on LTM. The second is system-level cost, capturing LLM usage in tokens and USD, CPU time, peak RAM, disk I/O and network usage.

Perspective: The experiment is conducted from the perspective of researchers developing infrastructure for the IoA, where independence from any commercial interest and applicability to DMAS and the IoA are central concerns.

Context: The experiment takes place on a simulated cloud-edge testbed in which agents communicate across a virtual network link whose bandwidth, latency, and jitter is programmatically fixed to reproduce realistic conditions.

Planning the Experiments

The planning phase covers selecting the context, formulating hypothesis, defining variables, designing the experiment, choosing the right instrumentation, and mitigating four classes of threats to validity.

Context Selection: The experiment is technology-oriented, operating on LLM-based agents rather than human subjects, is simulated and conducted offline¹. The benchmark comprises ten conversations replayed once in each framework and network condition. LLM temperature is fixed at zero and network shaping is controlled, which minimizes within-cell variance.

Hypothesis formulation: For RQ2, the null hypothesis $H_0^{(2)}$ states that the accuracy and cost of the LTM frameworks are not affected by the transition between network scenarios. The alternative hypothesis $H_1^{(2)}$ states that at least one framework exhibits a change in accuracy or cost. For RQ3, the null hypothesis $H_0^{(3)}$ states that no significant difference in the accuracy-cost trade-off exists between mem0, Graphiti and cognee. The alternative hypothesis $H_1^{(3)}$ states that at least one framework exhibits a significantly different trade-off.

Variables: Two independent variables are manipulated. The first is the LTM framework, with five levels, namely the three frameworks mem0, Graphiti, and cognee, and the two baselines RAG and full-context. The second is the network scenario, with two levels. The unconstrained scenario adds no artificial latency, jitter or bandwidth restrictions between edge and cloud components. The constrained scenario models a edge-to-cloud backhaul link, with 150 ms of latency, 512 KB/s of bandwidth, and 30 ms of jitter, representative of embedded edge devices accessing cloud services over a wide-area network [33].

The dependent variables fall into four groups. Latency consists of wall-clock time, isolated compute time, and any flush time incurred while persisting state to the memory backend. Resource utilization, recorded separately for the edge and cloud components, comprises CPU time, peak RAM, disk I/O, and network I/O. LLM usage and cost is logged as token counts and USD cost for edge-side and cloud-side calls, with the responder's context window consumption tracked as a separate line item. Answer quality, the fourth group, pairs each answer with its gold answer, records the number of memories returned by retrieval and the number of memory accesses, and is evaluated using an LLM-as-a-judge approach [34].

For each question, the judge is given the question, the gold answer from the benchmark, and the produced answer. It then returns a verdict based on the same prompt that was used to evaluate Graphiti due to its restrictiveness compared to the prompt used in mem0's benchmark [35], [36]. To mitigate sampling variance, the judge is invoked three times per question and the final verdict is taken by majority vote.

Experiment design: A full factorial design is adopted, combining each LTM framework with each network scenario, yielding ten experimental treatments. All ten conversations of the LoCoMo dataset are processed within each treatment.

Instrumentation: The testbed shown in Figure 1 simulates the DMAS on a single host using Docker Compose, with the edge-cloud scenarios implemented through three isolated Docker bridge networks rather than physical hosts. The edge-net hosts the coordinator and the LLM runtime ollama [37]

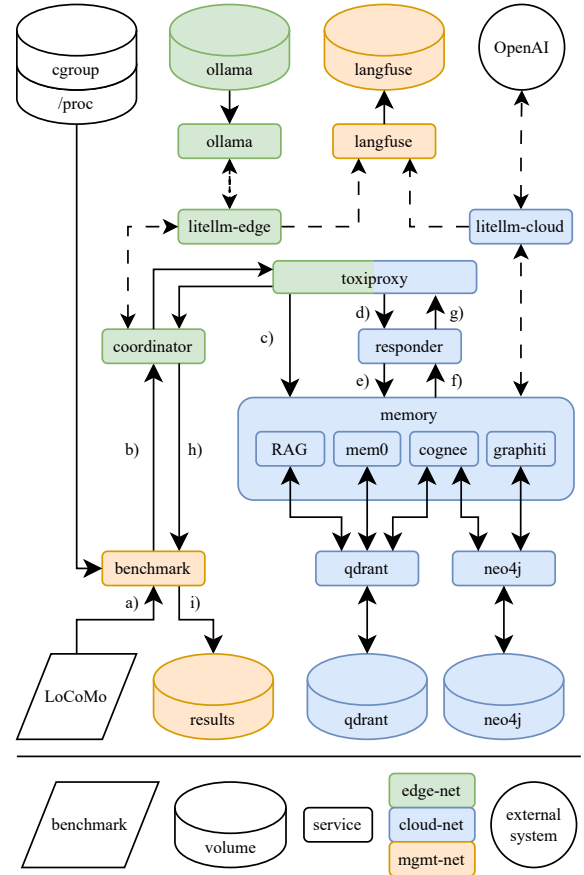


Figure 1. Testbed architecture including the benchmark, coordinator, responder and memory. LLM invocations to LiteLLM, Langfuse and OpenAI are marked as dashed lines. Components are split across an edge, cloud and management network. Toxiproxy is assigned to the edge and cloud networks.

for local small language models (SLMs). The cloud-net hosts the memory and responder services together with the two databases used by the LTM frameworks, namely qdrant used by mem0, cognee and the RAG baseline, and neo4j used by Graphiti and cognee. The mgmt-net bridge isolates the benchmark service and the langfuse observability stack [38] so that control-plane calls and metric scrapes never traverse the measured data plane. The role of Langfuse is to capture traces throughout each experiment. All LLM traffic is routed through the AI gateway LiteLLM [39], deployed as two instances reachable only from their own subnet, with litellm-edge transmitting to the local ollama instance and litellm-cloud dispatching to the OpenAI platform. LiteLLM is responsible for capturing the token usage, and converting it to USD. The only container in both data-plane subnets is toxiproxy [40], the gateway between edge-net and cloud-net that applies latency, jitter, and bandwidth limits to all edge-cloud traffic. While proc is used to track network traffic, cgroup captures CPU, RAM and disk I/O.

As indicated through the letters (a-g) in Figure 1, benchmark loads the LoCoMo benchmark from GitHub (a), decomposes each conversation into non-overlapping chunks

¹In Wohlin's framework, *offline* refers to a controlled experimental setting as opposed to a live industrial deployment.

of one turn, where one turn corresponds to a single message in the conversation, and forwards them sequentially to the coordinator (b), tagging every request with the target memory framework. The experiment proceeds in two phases. During the loading phase, the coordinator forwards each turn across the edge–cloud boundary through `toxiproxy` to the memory agent’s `/remember` endpoint (c), which routes the request to the active backend among RAG, `mem0`, `cognee`, and `Graphiti`. During the Q&A phase, the benchmark issues questions through the coordinator, which forwards each question across the same boundary to the responder’s `/ask` endpoint (d). The responder retrieves a fixed top $k = 20$ related memories from the memory agent over the intra-cloud path (e, f). The responder may search for memories in parallel up to four times. After retrieving all relevant memories, it generates the answer, which is returned through `toxiproxy` (g) and the coordinator (h) back to the benchmark. Finally, the benchmark appends all questions, answers, and metrics to a CSV file on disk (i).

The SLM *qwen2.5:3b-instruct-q4_K_M* using three billion parameters optimized for tool calling, drives the Q&A phase through function calls and is bypassed during the loading phase. The cloud LLM *gpt-4o-mini* is shared by every cloud-side call, namely the responder’s answer generation and the load-phase extraction performed by `mem0`, `Graphiti`, and `cognee`. Vector embeddings for `mem0`, `Graphiti`, `cognee`, and the RAG baseline are produced by OpenAI’s *text-embedding-3-small*.

Threats to Validity: Following Wohlin’s four-dimensional model [28], four classes of threats are considered.

For internal validity, all components involved are used at fixed versions. Memory backends are logically wiped between conversations while the testbed stays up, preserving the containers, networks, the `ollama` model cache, and the `neo4j` schema layer of indexes and uniqueness constraints.

During a dedicated warmup phase, an explicit initial call to the memory framework is issued to absorb any lazy initialization and first-invocation client overhead. Its resource counters and latency are excluded from the load and Q&A phases that follow. The `toxiproxy` toxics are set at the beginning of each conversation to match the configured network profile. To keep asynchronous work inside each call’s measurement window, `neo4j`’s `db.awaitIndexes` is invoked synchronously inside every `/memorize` handler so index population completes before the response returns, and the benchmark service holds its post-call `cgroup` snapshot until cloud-side disk I/O quiesces. This post-response wait is recorded separately from the actual computation as flush.

To minimize the risk that performance differences reflect researcher-introduced implementation choices rather than the frameworks themselves, the testbed code for `mem0` and `Graphiti` is adapted from their authors’ own repositories [41], [42]. A residual threat is that these benchmarks have been mutually disputed by competing framework providers, namely Zep has documented choices in `mem0`’s published evaluation that may understate `Graphiti`’s performance [43] and `mem0` has raised methodological concerns about Zep’s evaluation [44].

External validity is bounded by the choice of dataset, the underlying LLMs, and the fact that the cloud–edge separation is simulated on a single machine rather than physically distributed across two hosts. To probe whether observed differences depend on the specific framework set, the RAG and full-context baselines are included.

Construct validity is addressed by operationalizing answer correctness through an LLM-as-judge with majority voting over three independent calls, which mitigates mono-method bias in the verdict, and by operationalizing system-level cost as a multivariate vector of CPU, RAM, disk I/O, network, and token spend rather than a single proxy, so that no backend’s strengths can hide on an unmeasured axis. To further separate retrieval failure from reasoning error within the outcome variable, the responder’s system prompt instructs to not reply when the retrieved context is insufficient rather than to guess, so that absent memory is recorded as *unknown* instead of being miscoded as *wrong*. A residual threat is that the judge model shares a family with the responder, which may inflate agreement through self-enhancement bias. Majority voting attenuates but does not eliminate this effect [45].

Conclusion validity is based on 1,540 question–answer pairs across all ten LoCoMo benchmark conversations, replayed under both network scenarios and covering all questions in categories one through four. Following `mem0` and `zep`, category five questions, which have reference answers of the form “not mentioned”, are excluded as they conflate retrieval performance with the refusal behaviour of the responder LLM, rather than isolating the performance of the memory backend itself.

Operation

Preparation: The testbed is deployed on AWS EC2 `c7i.4xlarge` instances with 16 vCPUs, 32 GiB RAM, a 4th-generation Intel Xeon Scalable processor, and a 120 GB EBS `gp3` volume, running Ubuntu Server 26.04 LTS.

Execution: The experiment runs in parallel across four EC2 instances, one per framework setup (`cognee`, `mem0`, `graphiti` as well as RAG and full-context), each executing both network scenarios and writing its results to CSVs on disk.

Data validation: Each CSV is checked for the expected number of rows, missing fields, and anomalous resource readings. The validated dataset forms the input to the analysis in the next section, where the methodology applied to evaluate the results is described.

Analysis and Interpretation

Analysis and interpretation covers the statistical tests applied to the outcomes, the pricing model used to aggregate resource consumption into the total cost of ownership (TCO), and the dominance rule that summarizes the cost–accuracy trade-off.

The cost and accuracy for each backend are compared pairwise using the paired Wilcoxon signed-rank test [46], with the ten conversations from LoCoMo forming the pairing unit. A Bonferroni correction across the ten framework pairs sets the per-test significance threshold to $\alpha = 0.005$ [47]. The non-parametric form is preferred to a paired *t*-test because normality

Table I
AMAZON WEB SERVICES (AWS) AND OPENAI RESOURCES, SERVICES AND RATES USED FOR CALCULATING THE TCO OF EACH LTM AND BASELINE

Resource	Service	Rate
Disk	AWS Fargate (20 GB free)	\$0.000109/GB-hr
RAM	AWS Fargate (Linux/x86)	\$0.004445/GB-hr
vCPU time	AWS Fargate (Linux/x86)	\$0.04048/vCPU-hr
Cloud-to-edge egress	Amazon EC2 Data Transfer	\$0.09/GB
Edge-to-cloud ingress	Amazon EC2 Data Transfer	\$0.00/GB
Intra-cloud network	Amazon EC2 Data Transfer	\$0.00/GB
LLM input	OpenAI gpt-4o-mini	\$0.15/M tokens
LLM cached input	OpenAI gpt-4o-mini	\$0.075/M tokens
LLM output	OpenAI gpt-4o-mini	\$0.60/M tokens

Sources: AWS Fargate [48]; Amazon EC2 Data Transfer [49]; OpenAI [50].

of the per-conversation distributions cannot be assumed at $n = 10$. The same procedure pairs the constrained and unconstrained observations of each conversation in order to test for network sensitivity within each LTM.

TCO is computed as the sum of edge and cloud resource consumption per LTM framework across both the loading and Q&A phases, priced at the AWS Fargate `us-east-1` on-demand rates listed in Table I. Cloud-to-edge egress traffic between the `coordinator` and the `responder` would cross the public internet in a real deployment and is therefore charged at \$0.09 per GB. Edge-to-cloud ingress is charged at \$0.00 per GB. Traffic within `cloud-net` as well as egress traffic from `litellm-cloud` to OpenAI are also treated at zero cost.

The cost-accuracy trade-off is summarized as a statistical Pareto frontier over the five backends. For each ordered pair of backends (A, B) , A is said to dominate B if its TCO is strictly lower than B 's and its accuracy is either statistically equivalent to or higher than B 's, where statistical equivalence is taken as a failure to reject equality under the same paired Wilcoxon signed-rank test introduced above at $\alpha = 0.005$. The frontier is then defined as the set of backends not dominated by any other under this rule. The statistical relaxation of the standard dominance criterion prevents a marginal accuracy difference that the data cannot resolve from disqualifying a cheaper backend, and prevents a cheaper backend with significantly lower accuracy from being placed on the frontier.

IV. RESULTS

Two anomalies occur in the data. Firstly, the `responder` queried LTM multiple times for only 0.83% of questions across all conversations, frameworks, and network scenarios. To reduce ambiguity between experimental treatments, these questions are dropped across all LTM frameworks and baselines. Secondly, despite best efforts to eliminate this error through automated pruning, the LLMs context window was exceeded for 0.03% of all questions while running full-context. These questions are also removed from the results.

The cost of the warm-up and flush phases are not described further, as their impact does not further influence the results. Likewise, the paired Wilcoxon signed-rank test indicates no statistically significant difference between the unconstrained

and constrained network scenarios (all $p > 0.07$). Consequently, the unconstrained scenario is not considered in detail, since its exclusion does not affect the validity of the analysis.

The absence of statistically significant differences under artificially introduced latency, bandwidth constraints, and jitter further strengthens the robustness of the data from the constrained scenario, as it indicates that the reported results remain stable across varying network properties. While this limits differentiation between network scenarios, it yields repeated measurements under equivalent treatments, increasing the sample size and improving the reliability and robustness of the observed results across multiple experimental iterations.

The results are presented in four parts. The first part covers the loading phase, the second part the Q&A phase, the third part aggregates the cost of both phases into the TCO of each LTM, and the fourth parts calculates the cost-accuracy analysis based on the accuracy and TCO.

Loading Long-Term Memory

As shown in Table II, loading all ten conversations from LoCoMo required 24.11 hours, 32,719.68 MB of disk I/O in the cloud, 1.04 MB of disk I/O at the edge, 38.39 MB of edge-to-cloud traffic, and 27.07 MB of cloud-to-edge traffic. CPU time totaled 0.29 hours at the edge and 5.28 hours in the cloud. RAM usage peaked at 2,592.46 MB at the edge and 962.97 MB in the cloud. The OpenAI cost was \$11.64 USD.

Full-context was the fastest baseline, requiring only 0.61 hours to load all memories, followed by RAG at 0.93 hours, `mem0` at 4.44 hours, `cognee` at 8.44 hours, and Graphiti at 9.69 hours. Disk I/O was lowest for full-context at 1.23 MB, followed by RAG at 993.21 MB, `mem0` at 2,156.02 MB, Graphiti at 7,904.48 MB, and `cognee` at 21,664.74 MB. Network traffic between the `coordinator` and `responder` remained relatively stable across all frameworks, ranging from 13.08 MB for Graphiti to 13.28 MB for full-context when combining bidirectional traffic. CPU usage increased across systems, with full-context requiring the least total CPU time at 0.03 hours, followed by RAG at 0.10 hours, `mem0` at 0.32 hours, Graphiti at 0.67 hours, and `cognee` at 4.45 hours. Peak cloud RAM usage increased from full-context at 18.39 MB, to RAG at 70.01 MB, `mem0` at 126.50 MB, Graphiti at 866.93 MB, and `cognee` at 962.97 MB. OpenAI cost likewise increased from lowest to highest, with full-context at zero cost, followed by RAG at \$0.01 USD, `cognee` at \$1.32 USD, `mem0` at \$4.82 USD, and Graphiti at \$5.49 USD.

Answering Questions Using Long-Term Memory

The outcomes of the Q&A phase are further divided into two sections, namely cost and accuracy, capturing resource consumption and response correctness respectively.

Cost: After excision of the erroneous questions, answering 7,626 questions across all frameworks, baselines, and conversations in LoCoMo required 13.52 hours, 206.65 MB of disk I/O in the cloud, 0.08 MB of edge disk I/O, 8.01 MB of edge-to-cloud traffic, and 6.63 MB of cloud-to-edge traffic. CPU time totaled 26.46 hours at the edge and 1.29 hours in the cloud.

Table II

LATENCY AND SYSTEM-LEVEL COST OF THREE LTM FRAMEWORKS AND TWO BASELINES IN THE CONSTRAINED CLOUD-EDGE NETWORK SCENARIO. THE RAM PEAK SUBTOTALS AND TOTAL REFLECT THE HIGHEST PEAK IDENTIFIED DURING EXPERIMENTATION.

Phase	Memory	Latency (hours)	Cloud					Edge			
			CPU (hours)	RAM peak (MB)	Disk I/O (MB)	Network (MB)	OpenAI (USD)	CPU (hours)	RAM peak (MB)	Disk I/O (MB)	Network (MB)
Loading	cognee	8.44	4.36	962.97	21,664.74	5.48	\$1.32	0.09	2,562.79	0.92	7.69
	full-context	0.61	0.02	18.39	1.23	5.27	\$0.00	0.01	2,343.19	0.11	7.80
	Graphiti	9.69	0.56	866.93	7,904.48	5.39	\$5.49	0.11	2,591.10	0.01	7.65
	mem0	4.44	0.26	126.50	2,156.02	5.45	\$4.82	0.06	2,592.46	0.00	7.63
	RAG	0.93	0.08	70.01	993.21	5.48	\$0.01	0.02	2,342.22	0.00	7.62
	<i>Subtotal</i>	<i>24.11</i>	<i>5.28</i>	<i>962.97</i>	<i>32,719.68</i>	<i>27.07</i>	<i>\$11.64</i>	<i>0.29</i>	<i>2,592.46</i>	<i>1.04</i>	<i>38.39</i>
Q&A	cognee	2.93	0.80	1,001.63	176.07	1.22	\$1.05	5.20	2,841.20	0.00	1.58
	full-context	3.89	0.09	18.96	3.81	1.41	\$10.01	5.08	2,375.77	0.00	1.61
	Graphiti	2.76	0.27	876.25	17.46	1.27	\$1.00	5.25	2,856.97	0.07	1.62
	mem0	2.04	0.07	132.02	4.23	1.39	\$0.27	5.75	2,786.05	0.01	1.60
	RAG	1.90	0.06	79.76	5.08	1.34	\$0.40	5.18	2,501.20	0.00	1.60
	<i>Subtotal</i>	<i>13.52</i>	<i>1.29</i>	<i>1,001.63</i>	<i>206.65</i>	<i>6.63</i>	<i>\$12.73</i>	<i>26.46</i>	<i>2,856.97</i>	<i>0.08</i>	<i>8.01</i>
Total		37.63	6.57	1,001.63	32,926.33	33.70	\$24.37	26.75	2,856.97	1.12	46.40

Table III

ACCURATE, INCORRECT AND UNKNOWN RESPONSES ARE COMPUTED PER CONVERSATION AND THEN AVERAGED OVER ALL CONVERSATIONS. $\pm$ DENOTES THE STANDARD DEVIATION ACROSS ALL CONVERSATIONS.

Memory	Accurate (%)	Incorrect (%)	Unknown (%)
cognee	55.27 $\pm$ 3.60	16.55 $\pm$ 5.11	28.18 $\pm$ 4.92
full-context	77.16 $\pm$ 5.21	13.17 $\pm$ 3.50	9.67 $\pm$ 5.40
Graphiti	56.03 $\pm$ 5.56	18.39 $\pm$ 3.47	25.58 $\pm$ 3.38
mem0	81.08 $\pm$ 3.19	10.54 $\pm$ 2.64	8.38 $\pm$ 2.14
RAG	78.31 $\pm$ 5.75	10.56 $\pm$ 3.61	11.14 $\pm$ 3.95

RAM usage peaked at 2,856.97 MB at the edge and 1,001.63 MB in the cloud. The total cost of OpenAI’s LLM inference and embeddings was \$12.73 USD.

RAG was the fastest baseline for answering all questions, requiring 1.90 hours, followed by mem0 at 2.04 hours, Graphiti at 2.76 hours, cognee at 2.93 hours, and full-context at 3.89 hours. Disk I/O was lowest for full-context at 3.81 MB, followed by mem0 at 4.23 MB, RAG at 5.08 MB, Graphiti at 17.46 MB, and cognee at 176.07 MB. Network traffic was lowest for cognee at 2.80 MB, followed by Graphiti at 2.89 MB, RAG at 2.94 MB, mem0 at 2.99 MB, and full-context at 3.02 MB. Total CPU usage increased across systems, with mem0 requiring the least at 5.82 hours, followed by full-context at 5.17 hours, RAG at 5.24 hours, cognee at 6.00 hours, and Graphiti at 5.52 hours. Peak cloud RAM usage increased from full-context at 18.96 MB, to RAG at 79.76 MB, mem0 at 132.02 MB, Graphiti at 876.25 MB, and cognee at 1,001.63 MB. OpenAI cost ranged from lowest to highest, with mem0 at \$0.27 USD, RAG at \$0.40 USD, Graphiti at \$1.00 USD, cognee at \$1.05 USD, and full-context at \$10.01 USD.

Accuracy: The accuracy of each LTM is shown in Table III. Mem0 attains the highest performance, correctly answering 81.08% of questions, with 10.54% answered incorrectly and 8.38% marked as unknown. The RAG baseline follows with

Table IV

ACCURACY OF THREE LTM FRAMEWORKS AND TWO BASELINES ACROSS FOUR LoCoMo QUESTION CATEGORIES IN THE CONSTRAINED CLOUD-EDGE NETWORK SCENARIO. RESULTS ARE DESCRIPTIVE DUE TO THE LIMITED AMOUNT OF QUESTIONS PER CATEGORY.

Memory	Single-hop	Multi-hop	Temporal	Open-domain
cognee	52.79%	29.88%	43.14%	67.66%
full-context	67.94%	62.84%	36.65%	89.75%
Graphiti	56.00%	52.59%	31.83%	59.85%
Mem0	71.09%	74.79%	55.48%	88.81%
RAG	72.12%	73.08%	45.32%	85.30%

78.31% correct answers, 10.56% incorrect answers, and 11.14% unknown responses. The full-context baseline reaches 77.16% accuracy, with 13.17% incorrect and 9.67% unknown responses. Graphiti records 56.03% correct answers, with 18.39% incorrect and 25.58% unknown responses. Cognee achieves 55.27% correct, 16.55% incorrect and 28.18% unknown responses.

The accuracy of each LTM is further considered across four question categories as presented in Table IV: single-hop and multi-hop depending on whether one or multiple retrieval steps are required, temporal questions depending on whether time-based reasoning is required, and open-domain questions depending on whether questions span broad topics or a restricted setting. Per-category accuracy is reported as indicative trends only, since the per-conversation per-category sample size is too small to support statistical significance testing. Multi-hop questions show the largest variation between systems, ranging from 74.79% for mem0 and 73.08% for RAG down to 29.88% for cognee. Open-domain questions are the easiest overall, with no framework or baseline achieving less than 59.85% accuracy.

Total Cost of Ownership

Based on the unit prices provided in Table I, the TCO reflects both system-level and LLM-related expenditures aggregated

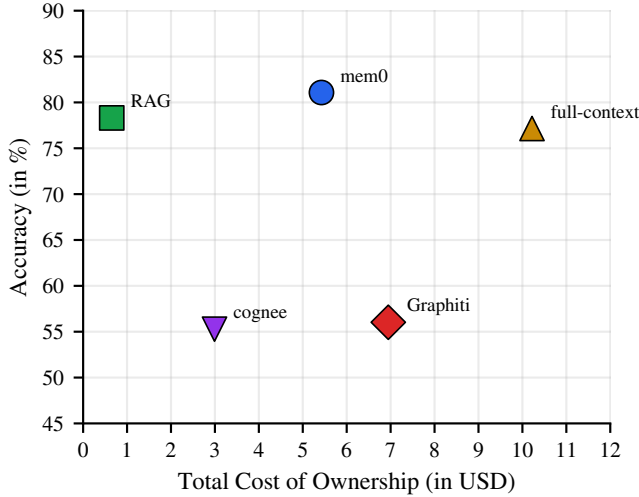


Figure 2. Cost-accuracy trade-off of three LTM frameworks and two baselines in the constrained network scenario. RAG and mem0 are pareto-optimal.

across the loading and Q&A phases, covering both cloud and edge resources. RAG has the lowest TCO at \$0.65 USD, followed by cognee at \$2.99 USD, mem0 at \$5.43 USD, Graphiti at \$6.95 USD, and full-context at \$10.22 USD. Given that disk usage remains below 20 GB throughout all experiments, storage-related costs are effectively zero under the applied pricing model.

Cost-Accuracy Trade-Off

Figure 2 shows that only RAG and mem0 lie on the Pareto frontier. RAG attains a mean accuracy of 78.31% at a TCO of \$0.65 USD. Mem0 reaches 81.08% at a TCO of \$5.43 USD. The full-context baseline records 77.16% at a TCO of \$10.22 USD. Graphiti scores 56.03% at a TCO of \$6.95 USD, and cognee records 55.27% at a TCO of \$2.99 USD.

V. DISCUSSION

The following section interprets the empirical findings in relation to the three research questions and four hypotheses by examining the mechanisms underlying the observed differences in retrieval accuracy, latency sensitivity, and system-level cost across the evaluated LTMs and baseline configurations.

As shown in Figure 3, accuracy separates into two statistically distinct clusters: Mem0, RAG, and full-context span 77.16% to 81.08%, while Graphiti and cognee reach only 56.03% and 55.27% respectively. All cross-cluster comparisons are significant at $p = 0.002$ under a paired Wilcoxon signed-rank test with Bonferroni correction, and no within-cluster differences reach significance (mem0 versus RAG: $p = 0.084$, Graphiti versus cognee: $p = 0.845$). The ranking is stable across all ten LoCoMo conversations.

The gap is driven by retrieval incompleteness rather than reasoning failure. As presented in Table III, Graphiti and cognee return unknown responses on 25.58% and 28.18% of questions respectively, compared to 8.38% to 11.14% for the upper cluster,

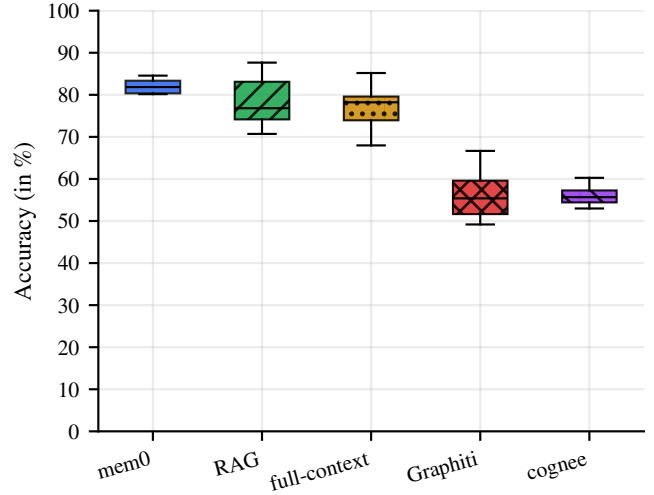


Figure 3. Accuracy of three LTM frameworks and two baselines across ten LoCoMo conversations in the constrained cloud-edge network scenario. Two clusters emerge: the first one containing mem0, RAG and full-context, and the second one containing graphiti and cognee.

and a paired Wilcoxon test on unknown rates reproduces the same structure at $p = 0.002$. When graph-based and hybrid LTM do retrieve context, their incorrect answer rates are only moderately higher than those of the upper cluster. The dominant failure mode is therefore the absence of retrieved content, not its misinterpretation. Graph-based ingestion introduces entity resolution, relation labeling, and schema-constrained extraction, each an opportunity for information loss before retrieval is attempted. This compounds on multi-hop questions, where the cross-cluster gap is largest, ranging from 74.79% for mem0 down to 29.88% for cognee.

Counterintuitively, full-context scores below mem0, which supplies only a small set of extracted memories. Presenting a large undifferentiated prompt dilutes relevant facts among irrelevant turns. Compression precision therefore determines accuracy more than context volume, and the full-context result represents the ceiling achievable without compression, a ceiling that selective extraction exceeds.

Only RAG and mem0 are non-dominated on the statistical Pareto frontier. RAG matches top-cluster accuracy at a TCO of \$0.65, requiring no LLM calls during loading and no graph infrastructure. Mem0 achieves the highest observed accuracy (81.08%) at \$5.43, below both full-context (\$10.22) and Graphiti (\$6.95). Full-context is dominated simultaneously on cost and accuracy. Graphiti and cognee are dominated on accuracy at comparable or higher cost. For cost-sensitive IoA deployments, RAG is the pragmatic default, and mem0 is appropriate where accuracy gains justify the expenditure.

Accuracy and unknown rate are invariant to network constraints for every backend (all $p > 0.07$, $\alpha = 0.01$). The memory index content, established during loading, determines what the agent can answer, and the link over which memories are subsequently fetched does not. A statistically significant latency

increase of 4% to 5% is observed for mem0 and RAG under the constrained scenario (both $p = 0.006$), as both retrieve context across the edge-cloud link at query time. Full-context, Graphiti, and cognee show no significant latency response, their processing being compute-bound at the tested constraint level. Link quality governs what a DMAS pays in latency, but not what it can remember.

These results diverge substantially from both vendor-published evaluations, illustrating the evaluation bias identified in Section I. Mem0’s published benchmark reports base accuracy of 66.88% on LoCoMo [13], versus 81.08% found here. Zep’s counter-evaluation reports Graphiti at 75.14% and mem0 at 66.88% [43], directionally opposite to the present findings. The testbed here uses evaluation code from both authors’ repositories and the Graphiti-derived judge prompt, which should if anything favor Graphiti. Despite this, Graphiti’s 25.58% unknown rate persists across all ten conversations, pointing to indexing incompleteness as the cause rather than any evaluator artefact. The mutual disparity confirms that a shared harness, judge, infrastructure, and dataset are necessary for cross-framework comparison, and that neither vendor benchmark satisfies all four simultaneously.

RQ1 is answered by the testbed presented in Section III. Reproducible LTM comparison in a DMAS requires network isolation across separated subnets, deterministic network shaping, unified LLM routing and observability for normalized cost attribution, full-system resource instrumentation, and a majority-voted LLM-as-a-judge with a shared prompt and zero LLM temperature across all frameworks. A coordinator further enforces the order of treatments, and structures their outcomes.

In response to **RQ2**, $H_0^{(2)}$ cannot be rejected. Accuracy and unknown rate are statistically invariant to network conditions for all frameworks and baselines. This indicates that memory quality is primarily determined at ingestion time rather than at retrieval time. The transition to a constrained cloud-edge DMAS introduces only a modest, architecture-dependent latency increase, affecting systems that perform query-time retrieval but leaving correctness unchanged. In this setting, network conditions act as a throughput constraint rather than an information constraint.

Regarding **RQ3**, $H_0^{(3)}$ is rejected ($p = 0.002$ for all cross-cluster comparisons). RAG and mem0 are the only non-dominated systems on the joint cost-accuracy frontier. The dominance structure is primarily driven by retrieval coverage: systems with higher effective recall at ingestion or indexing time consistently outperform those relying on richer structural representations that are not fully recovered at query time. RAG offers the most cost-efficient configuration, while mem0 provides the highest observed accuracy, with both occupying distinct optimal points on the cost-accuracy trade-off curve.

Limitations

This section outlines the four key limitations of the study, including constraints in benchmark coverage, system selection, testbed design, and ingestion granularity that bound the generalizability of the results.

A key limitation of this study is the reliance on the LoCoMo benchmark without evaluation across additional LTM benchmarks, despite prior work identifying several methodological shortcomings. LongMemEval and ES-MemEval note that LoCoMo does not adequately evaluate reasoning over updated user states and lacks conflict detection, abstention behavior, and explicit user modeling [5], [20]. MobileMem highlights its single-session structure, limiting multi-session goal continuity, while MemoryAgentBench distinguishes long-context evaluation from true LTM, where memory should be a compressed and selective representation rather than a full transcript [51], [52]. Zep further critiques its limited difficulty due to context windows that allow strong full-context baselines. Additional issues include incomplete ground truth, multimodal inconsistencies, and ambiguous questions [43].

A second limitation in the comparison of LTM is the lack of intra-class evaluation, where only a single representative system is included for each category. In particular, the study does not include multiple vector-based, graph-based, and hybrid LTM frameworks, limiting the ability to assess variance within each architectural class or to determine whether observed effects are method-specific or representative of the broader class behavior.

A third limitation concerns the design of the testbed. The system is deployed on a single machine inside Docker, where all components share the same underlying computational resources. The responder acts as a central point through which each Q&A query from the coordinator is routed, making the system a quasi-distributed MAS rather than a DMAS by definition. Although bandwidth constraints, increased latency, and injected jitter were introduced to approximate a DMAS environment, the resulting differences between constrained and unconstrained scenarios were not statistically significant.

A fourth limitation concerns the evaluation of knowledge ingestion strategies. The study focuses on ingestion at the granularity of individual conversational turns, which reflects episodic interaction data but does not cover alternative ingestion approaches such as long-form documents or large unstructured corpora. This is particularly relevant for cognee, which is designed to operate on extensive inputs including full documents and large datasets rather than isolated dialogue segments.

Future Work

In the field of LTM, future work could explore the deliberate fragmentation or partial removal of stored memory after distribution, either in vector space representations or within segments of knowledge graphs, to evaluate how effectively LTM systems can recover, reconstruct, or infer missing information. This would provide a more direct measure of robustness under incomplete memory conditions and help characterize the resilience of retrieval and reasoning mechanisms when confronted with partial or corrupted memory states.

Regarding DMAS and the IoA, future work could involve constructing an IoA that integrates agent discovery, identity management, distributed architectures, edge deployments, and LTM across physically distributed hardware. Experimental conditions should reflect real-world network characteristics,

including measured latency, bandwidth constraints, and jitter, rather than simulated scenarios. Following established practices in distributed systems research, memory could be partitioned across agents using strategies such as random assignment, batching, or round-robin distribution, enabling systematic analysis of memory consolidation mechanisms across distributed components, including how distributed LTM is merged, reconciled, and maintained consistently across agents.

Future studies of LTM in DMAS and the IoA could further incorporate dynamic and adversarial conditions, including agents joining and leaving the network, and Byzantine fault tolerance scenarios. This would enable a more rigorous evaluation of robustness, consistency, and resilience of memory in DMAS and the IoA under realistic and adversarial deployment settings.

Conclusion

This paper presents an independent reproducible testbed for evaluating LTM frameworks in a simulated cloud-edge DMAS, benchmarking mem0, Graphiti, and cognee alongside RAG and full-context baselines across ten LoCoMo conversations under unconstrained and constrained network scenarios.

Accuracy separates into two statistically distinct clusters ($p = 0.002$, paired Wilcoxon, Bonferroni-corrected). Mem0, RAG, and full-context reach 77% to 81%, while Graphiti and cognee reach only 55% to 56%, a gap driven by retrieval incompleteness rather than reasoning failure. Compression precision determines accuracy more than context volume, as full-context forwarding underperforms mem0's selective extraction despite supplying the entire conversation per question. Network constraints leave retrieval quality unchanged, imposing a modest latency penalty on vector backends.

On the cost-accuracy frontier, only RAG and mem0 are non-dominated. RAG matches the upper cluster at 8.4 times lower TCO than mem0, making it the pragmatic default for cost-sensitive IoA deployments. Mem0 is preferable where maximum accuracy justifies the additional expenditure.

ACKNOWLEDGMENT

The authors thank Prof. Emil Björnson for his mentorship during the writing of a university report that led to this publication, and Aleksey Veresov for his review of the manuscript. ChatGPT and GPT-5.2 were used to generate the code of the testbed. Claude Code and Opus 4.7 were used to extend the code of the testbed based on the reviewer's comments. Cursor and Claude Sonnet 4.6 were used to generate the interactive Python notebook used to analyze the results. GPT-5.3-mini, GPT-5.5, Claude (Web) and Claude Opus 4.7 were used to incorporate the reviewer's comments into the manuscript.

REFERENCES

- [1] Z. Zhang, Q. Dai, X. Bo, C. Ma, R. Li, X. Chen, J. Zhu, Z. Dong, and J.-R. Wen, "A Survey on the Memory Mechanism of Large Language Model-based Agents," *ACM Transactions on Information Systems*, vol. 43, no. 6, pp. 1–47, Nov. 2025. [Online]. Available: <https://dl.acm.org/doi/10.1145/3748302>
- [2] D. Zhang, Y. Feng, H. Liu, C. Sun, J. Luo, X. Chen, and X. Li, "Conversational Agents: From RAG to LTM," in *Proceedings of the 2025 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, ser. SIGIR-AP 2025. New York, NY, USA: Association for Computing Machinery, Dec. 2025, pp. 447–452. [Online]. Available: <https://dl.acm.org/doi/10.1145/3767695.3769671>
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf
- [4] A. Maharana, D.-H. Lee, S. Tulyakov, M. Bansal, F. Barbieri, and Y. Küng, "Evaluating very long-term conversational memory of llm agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024, pp. 13 851–13 870.
- [5] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, "LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=pZiyCaVuti>
- [6] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, "A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges," *Vicinagearth*, vol. 1, no. 1, p. 9, Oct. 2024. [Online]. Available: <https://link.springer.com/10.1007/s44336-024-00009-2>
- [7] E. T. C. Chagas, A. Houmansadr, F. G. Guimarães, and H. S. Ramos, "Distributed Multi-Agent AI Systems: Scalability, Challenges, and Applications," in *2025 IEEE 45th International Conference on Distributed Computing Systems Workshops (ICDCSW)*, Jul. 2025, pp. 412–415, iSSN: 2332-5666. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11262644>
- [8] M. Rojas-Contreras and J. A. C. Jaimes, "Architectural Design of Decentralized Multi-agent Systems: The Agent Squad Framework Approach," in *Applications of Computational Intelligence*, A. D. Orjuela-Cañón, J. A. Lopez, and O. J. Suarez, Eds. Cham: Springer Nature Switzerland, 2026, pp. 147–161.
- [9] W. Chen, Z. You, R. Li, y. guan, C. Qian, C. Zhao, C. Yang, R. Xie, Z. Liu, and M. Sun, "Internet of Agents: Weaving a Web of Heterogeneous Agents for Collaborative Intelligence," in *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, Eds., vol. 2025, 2025, pp. 36 374–36 411. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/file/59c27bf8d56d3d50c7aeaf7535dee975-Paper-Conference.pdf
- [10] Z. Aminiranjbar, J. Tang, Q. Wang, S. Pant, and M. Viswanathan, "DAWN: Designing Distributed Agents in a Worldwide Network," *IEEE Access*, vol. 13, pp. 138 795–138 812, 2025. [Online]. Available: <https://ieeexplore.ieee.org/document/11078243/>
- [11] Y. Wang, S. Guo, Y. Pan, Z. Su, F. Chen, T. H. Luan, P. Li, J. Kang, and D. Niyato, "Internet of Agents: Fundamentals, Applications, and Challenges," *IEEE Transactions on Cognitive Communications and Networking*, vol. 12, pp. 4476–4501, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11207716>
- [12] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," Feb. 2024, arXiv:2310.08560 [cs]. [Online]. Available: <http://arxiv.org/abs/2310.08560>
- [13] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," Apr. 2025, arXiv:2504.19413 [cs]. [Online]. Available: <http://arxiv.org/abs/2504.19413>
- [14] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory," Jan. 2025, arXiv:2501.13956 [cs]. [Online]. Available: <http://arxiv.org/abs/2501.13956>
- [15] C. Latimer, N. Boschi, A. Neeser, C. Bartholomew, G. Srivastava, X. Wang, and N. Ramakrishnan, "Hindsight is 20/20: Building Agent Memory that Retains, Recalls, and Reflects," Dec. 2025, arXiv:2512.12818 [cs]. [Online]. Available: <http://arxiv.org/abs/2512.12818>

- [16] V. Markovic, L. Obradovic, L. Hajdu, and J. Pavlovic, "Optimizing the Interface Between Knowledge Graphs and LLMs for Complex Reasoning," May 2025, arXiv:2505.24478 [cs]. [Online]. Available: <https://arxiv.org/abs/2505.24478>
- [17] S. Sarin, L. Singh, B. Sarmah, and D. Mehta, "Memoria: A Scalable Agentic Memory Framework for Personalized Conversational AI," in *2025 5th International Conference on AI-ML-Systems (AIMLSys)*. Bangalore, India: IEEE, Oct. 2025, pp. 32–39. [Online]. Available: <https://ieeexplore.ieee.org/document/11330332/>
- [18] M. Mohammadi, Y. Li, J. Lo, and W. Yip, "Evaluation and Benchmarking of LLM Agents: A Survey," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, ser. KDD '25. New York, NY, USA: Association for Computing Machinery, Aug. 2025, pp. 6129–6139. [Online]. Available: <https://dl.acm.org/doi/10.1145/3711896.3736570>
- [19] S. Guan, J. Wang, J. Bian, B. Zhu, J.-G. Lou, and H. Xiong, "Evaluating LLM-based Agents for Multi-turn Conversations: A Survey," *ACM Transactions on Intelligent Systems and Technology*, vol. 17, no. 4, pp. 1–40, Aug. 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/3793671>
- [20] T. Chen, J. Lu, Y. Shen, and L. Zhang, "ES-MemEval: Benchmarking Conversational Agents on Personalized Long-Term Emotional Support," in *Proceedings of the ACM Web Conference 2026*. Dubai United Arab Emirates: ACM, Apr. 2026, pp. 5810–5821. [Online]. Available: <https://dl.acm.org/doi/10.1145/3774904.3792143>
- [21] Y. Du, B. Wang, Y. He, B. Liang, B. Wang, Z. Li, L. Gui, J. Z. Pan, R. Xu, and K.-F. Wong, "MemGuide: Intent-Driven Memory Selection for Goal-Oriented Multi-Session LLM Agents," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 36, pp. 30584–30592, Mar. 2026. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/40313>
- [22] S. Guo, Y. Wang, Z. Su, Y. Pan, Q. Hu, and T. H. Luan, "Agent Discovery in Internet of Agents: Challenges and Solutions," *IEEE Network*, pp. 1–9, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11427326>
- [23] A. Aydeger, E. Zeydan, J. Manges-Bafalluy, Y. Turk, S. A. Khowaja, and K. Dev, "Decentralized Digital Identity Management for Large Language Model Agents," *IEEE Communications Standards Magazine*, vol. 10, no. 1, pp. 87–94, Mar. 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11367791>
- [24] Y. Ding, A. Twabi, J. Yu, L. Zhang, T. Kondo, and H. Sato, "Decentralized Multi-Agent System with Trust-Aware Communication," in *2025 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA)*, Oct. 2025, pp. 1439–1445, iSSN: 2158-9208. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11245224>
- [25] H. Mohammed, H. Yin, and S. C. Boyapati, "Context Adaptive Memory-Efficient LLM Inference for Edge Multi-Agent Systems," in *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, ser. AAMAS '25. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2025, pp. 2678–2680.
- [26] X. Wang, J. He, Z. Tang, J. Guo, J. Lou, L. Qian, T. Wang, and W. Jia, "Adaptive AI Agent Placement and Migration in Edge Intelligence Systems," in *2025 IEEE 25th International Conference on Communication Technology (ICCT)*, Oct. 2025, pp. 1475–1480, iSSN: 2576-7828. [Online]. Available: <https://ieeexplore.ieee.org/document/11373977>
- [27] G. Tadi and A. Mittal, "Performance and Cost Optimization of Federated LLM Agents in Edge Computing Environments," in *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*, Nov. 2025, pp. 1267–1274. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/11391052>
- [28] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2024. [Online]. Available: <https://link.springer.com/10.1007/978-3-662-69306-3>
- [29] V. R. Basili, G. Caldiera, and H. D. Rombach, "The Goal Question Metric Approach," 1994. [Online]. Available: <https://api.semanticscholar.org/CorpusID:13884048>
- [30] T. Singh, "Mem0 raises \$24M to build the memory layer for AI," Nov. 2025. [Online]. Available: <https://mem0.ai/series-a>
- [31] Crunchbase Inc., "Zep AI - Crunchbase Company Profile & Funding," May 2026. [Online]. Available: <https://www.crunchbase.com/organization/zep-ai>
- [32] V. Markovic, "Cognee - Cognee Raises \$7.5M Seed to Build Memory for AI Agents," Feb. 2026. [Online]. Available: <https://www.cognee.ai/blog/cognee-news/cognee-raises-seven-million-five-hundred-thousand-dollars-seed>
- [33] 3rd Generation Partnership Project, "3rd Generation Partnership Project; Technical Specification Group Services and System Aspects; Service requirements for the 5G system; Stage 1 (Release 20)," Mar. 2026. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-k60.zip
- [34] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Z. Lin, B. Zhang, L. Ni, W. Gao, Y. Wang, and J. Guo, "A survey on LLM-as-a-judge," *The Innovation*, p. 101253, Jan. 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S2666675825004564>
- [35] P. Rasmussen, "zep-papers/kg_architecture_agent_memory/locomo_eval/zep_locomo_eval.py," May 2025. [Online]. Available: https://github.com/getzep/zep-papers/blob/4b7f26cc76cca20743314ba9acb8c2cb6adc42f6/kg_architecture_agent_memory/locomo_eval/zep_locomo_eval.py
- [36] S. Rathi, "memory-benchmarks/benchmarks/locomo/prompts.py," Apr. 2026. [Online]. Available: <https://github.com/mem0ai/memory-benchmarks/blob/f75666d33ef560f0f196746e0e16c515d17e6856/benchmarks/longmemeval/prompts.py>
- [37] Ollama, "ollama/ollama," May 2026. [Online]. Available: <https://github.com/ollama/ollama>
- [38] Langfuse GmbH and / Finto Technologies Inc., "langfuse/langfuse," May 2026. [Online]. Available: <https://github.com/langfuse/langfuse>
- [39] LiteLLM, "BerriAI/litellm," May 2026. [Online]. Available: <https://github.com/BerriAI/litellm>
- [40] Shopify, "toxiproxy," Nov. 2025. [Online]. Available: <https://github.com/Shopify/toxiproxy>
- [41] C. Kumar, "mem0ai/memory-benchmarks," May 2026. [Online]. Available: <https://github.com/mem0ai/memory-benchmarks>
- [42] P. Rasmussen and D. Chalef, "getzep/zep-papers," Apr. 2026. [Online]. Available: <https://github.com/getzep/zep-papers>
- [43] D. Chalef and P. Rasmussen, "Is Mem0 Really SOTA in Agent Memory?" May 2025. [Online]. Available: <https://blog.getzep.com/lies-damn-lies-statistics-is-mem0-really-sota-in-agent-memory/>
- [44] D. Yadav, "Revisiting Zep's 84% LoCoMo Claim: Corrected Evaluation & 58.44% Accuracy · Issue #5 · getzep/zep-papers," May 2025. [Online]. Available: <https://github.com/getzep/zep-papers/issues/5>
- [45] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 46595–46623. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/91f18a1287b398d378ef22505bf41832-Paper-Datasets_and_Benchmarks.pdf
- [46] F. Wilcoxon, "Individual Comparisons by Ranking Methods," *Biometrics Bulletin*, vol. 1, no. 6, p. 80, Dec. 1945. [Online]. Available: <https://www.jstor.org/stable/10.2307/3001968?origin=crossref>
- [47] C. E. Bonferroni, *Teoria statistica delle classi e calcolo delle probabilità*. Seeber, 1936. [Online]. Available: <https://books.google.se/books?id=3CY-HQAACAAJ>
- [48] Amazon Web Services, Inc., "AWS Fargate Pricing," 2026. [Online]. Available: <https://aws.amazon.com/fargate/pricing/>
- [49] —, "EC2 On-Demand Instance Pricing," 2026. [Online]. Available: <https://aws.amazon.com/ec2/pricing/on-demand/>
- [50] OpenAI Developers, "Pricing | OpenAI API," May 2026. [Online]. Available: <https://developers.openai.com/api/docs/pricing>
- [51] X. Deng, Y. Xue, Y. Chen, M. Mao, R. Zhong, B. Xu, J. Fang, H. Xu, T. Wu, Y. Xu, S. Deng, H. Wang, H. Chen, and N. Zhang, "MobileMem: Evaluating Long-Horizon Memory for Language Agents in Real-World Mobile Environments," in *ICLR 2026 Workshop on Lifelong Agents: Learning, Aligning, Evolving*, 2026. [Online]. Available: <https://openreview.net/forum?id=w5I1HrMgJ>
- [52] Y. Hu, Y. Wang, and J. McAuley, "Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions," 2025, version Number: 3. [Online]. Available: <https://arxiv.org/abs/2507.05257>